T.P. Parallélisme : PVM - Elimination de Gauss

A rendre: compte-rendu présentant analyses, algorithmes, résultats

1 Environnement PVM, programmation SPMD

1. Configurer `ssh` sur votre compte pour réduire les saisies de mot de passe lors d'un login d'une machine sur une autre (voir `http://www.esiee.fr/~perrotol/ssh-guide.html`)
2. Editer le fichier `~/.cshrc` et rajouter à la fin la ligne : `source /nfs/opt/pvm/pvm/cshrc.pvmpi`

   Editer le fichier `~/.bashrc` et rajouter à la fin la ligne : `. /nfs/opt/pvm/pvm/bashrc.pvmpi`
3. Créez les répertoires `$HOME/pvm3/bin/LINUX64/` pour les binaires et placez vos sources dans `$HOME/pvm3/`
4. Copier depuis `/nfs/opt/pvm/pvm3/` les sources `hello.c`, `tokenring-sibling.c`, `gauss.c` et `Makefile` dans `~/pvm3`. Compiler (`make tokenring-sibling`) et vérifier que `pvm` et `xpvm` fonctionnent correctement en les exécutant.

2 Elimination de Gauss

Le but du TP est de proposer une parallélisation de l'élimination de Gauss avec PVM. L'élimination de Gauss est un algorithme de triangularisation de matrice qui peut être utilisé pour trouver les solutions d'un système d'équations linéaires du type $A.X = B$ où A est une matrice $n \times n$ et X et B deux vecteurs de taille n. L'algorithme consiste à annuler la partie triangulaire inférieure de la matrice en itérant $n-1$ étapes ($k = 0, \ldots n-2$) qui annulent les éléments $i = k+1, \ldots n-1$ de la colonne k de la matrice par substitution de la ligne k avec les lignes $i = k+1, \ldots n-1$.

$$A = \begin{pmatrix} a_{00} & \cdots & & \cdots & a_{0n-1} \\ & \ddots & & & \vdots \\ & & a_{kk} \cdots a_{kj} \cdots & & a_{kn-1} \\ \vdots & & \vdots \quad\quad \vdots & & \\ & & a_{ik} \cdots a_{ij} & & \vdots \\ & & \vdots & & \\ a_{n-10} & \cdots & a_{n-1k} & \cdots & a_{n-1n-1} \end{pmatrix}$$

```
FOR k = 0 TO n-2
     FOR i = (k+1) TO n-1
          FOR j = (k+1) TO n-1
               a[i][j] = a[i][j] - a[i][k]/a[k][k] x a[k][j]
```

3 Méthodologie

1. Réfléchir à la **distribution des données sur les processeurs**: quels sont les choix de répartition de la matrice les schémas et coûts de communications engendrés. On suppose pour simplifier que **la taille de la matrice est divisble par le nombre de processeurs**.
2. Copier le source `tokenring-sibling.c` en `gaussp.c`. A partir du programme d'élimination de gauss séquentiel `gauss.c`, fabriquer le squelette de votre programme paralléle : le `main()` de `gauss.c` devient `dowork()` de `gaussp.c`

   Faites des liens symboliques des matrices d'exemple à la racine de votre compte : `ln -s /nfs/opt/pvm/pvm3/m* ~/`
3. Commencez par paralléliser le **chargement** (`matrix_load()`) et la **sauvegarde** (`matrix_save()`) du fichier de la matrice en incluant sa **diffusion** et sa **concentration** sur les processeurs.

   Valider les procédures développées et observer les communications avec `xpvm` avec les matrices d'exemples.

   Réfléchissez à la possibilité d'utiliser des accès fichiers concurents.
4. Paralléliser le calcul de l'élimination de Gauss.
5. Prenez un nombre de tâches inférieur ou égal à n. Commencez par valider votre programme avec `xpvm` et peu de tâches (2 ou 4) sur des matrices de petite tailles (4, 8, 16).
6. Faites des mesures de Speedup et d'Efficacité en fonction du nombre de machines sur des matrices de taille plus importantes (500, 1000, 5000 et 10000) et sur des réseaux différents (salle 500x, 510x,...)

4 Remarques, conseils, idées...

1. **Ne pas inclure les accès fichiers dans les mesures de temps**; l'objectif du TP n'est pas un benchmark du système de fichier, mais de votre algorithme de calcul parallèle.
2. Ne **surtout pas sauver la matrice résultat** lors des mesures de Speedup, surtout sur les grosses matrices.
3. Faut il utiliser `pvm` ou `xpvm` pour faire les mesures de speedup ? Faut il faire des `printf()` ?
4. Est il possible d'améliorer les performances en faisant du recouvrement calculs / communications ? Si oui, faites des mesures avant / après.
5. Que se passe-t-il lorsqu'un pivot est nul ?

5 Bug du montage des systèmes de fichiers ESIEE (clés SSH innacessibles)

Les répertoires des comptes utilisateurs sont montés dans ~/ **au moment de l'authentification du login**. Ce qui rend impossible l'utilisation normale des clés ssh (dans ~/.ssh/) pour se loguer d'une machine à une autre. Or PVM utilise ce mécanisme pour lancer des taches sur un cluster de machines.

Pour pouvoir lancer des taches sur une autre machine, vous devez vous loguer UNE FOIS au début du TP sur les PCs cibles en tapant votre mot de passe, et laisser la connexion active pour conserver le montage du repértoire ~/ sur la machine distante. Ceci garanti que PVM trouvera les fichiers binaires à executer.

Cela peut se faire en ouvrant un terminal spécifiquement à cet usage, dans lequel vous allez vous loguer successivement sur toutes les machines que vous voulez placer dans le cluster PVM avec `ssh pc5201a` . Vous minimisez ce terminal et vous le gardez actif.

ATTENTION de ne pas fermer ce terminal par erreur, il faudrait refaire la manip !

Malheureusement, les droits d'accès du système de fichier monté étant mal configurés, les clés SSH restent innaccessibles et il faudra taper votre mot de passe pour ajouter les machines dans PVM (commande `pvm> add pc5201a`).

6 Annexes : contenu du .bashrc.pvmpi

Fichier bashrc.pvmpi :

```
# .bashrc for pvm and xpvm at ESIEE

export PVM_ROOT=/nfs/opt/pvm/pvm/pvm

export PVM_ARCH=$($PVM_ROOT/lib/pvmgetarch)
export PVM_RSH=$(which ssh)

export MPI_ROOT=/nfs/opt/pvm/mpich2-1.4.1-log

if [ -z "$MANPATH" ]; then
   export MANPATH=/usr/share/man:$PVM_ROOT/man:${MPI_ROOT}/share/man
else
   export MANPATH=$MANPATH:$PVM_ROOT/man:${MPI_ROOT}/share/man
fi

export XPVM_ROOT=${PVM_ROOT}/xpvm

export PATH=$PVM_ROOT/lib:$PVM_ROOT:$XPVM_ROOT/src/$PVM_ARCH:$MPI_ROOT/bin:$PATH
```